



HPCN: Avaliación da escalabilidade de kernels de Dinámica de Fluídos Computacional (CFD, Computational Fluid Dynamics) en clusteres virtuais.

Estudante: Omar Montenegro Macía

Dirección: Xoán Carlos Pardo Martínez

A Coruña, decembro de 2024.

Índice Xeral

1 Configuración experimental	2
1.1 Restricciones del laboratorio	2
1.2 Kernel comunicaciones	2
1.2.1 Cálculo de costes	3
1.2.2 Despliegue	4
1.3 Kernel computación	4
1.3.1 Cálculo de costes	6
1.3.2 Despliege	7
2 Resultados del experimento	9
2.1 Métricas para cada kernel	9
2.2 Factores primarios:	9
2.3 Resultados del kernel de comunicación CG	9
2.3.1 Tiempo promedio	9
2.3.2 Speed Up	10
2.3.3 Eficiencia paralela	11
2.3.4 Análisis	11
2.4 Resultados kernel de computación MG	14
2.4.1 Tiempo promedio	14
2.4.2 Speed Up	15
2.4.3 Eficiencia paralela	15
2.4.4 Análisis	16
2.5 Conclusiones	19
Bibliografía	21

Configuración experimental

1.1 Restricciones del laboratorio

Antes de realizar el experimento, se ha consultado la documentación referente a las limitaciones de los servicios de AWS academy, concretamente del servicio EC2 [1].

La principal limitación que recorta la escalabilidad del experimento es que AWS academy soporta tamaños de instancia hasta máximo el tipo **large**. Por lo que todas las instancias de este experimento serán de este tipo.

Otra limitación importante es el nº de instancias corriendo al mismo tiempo. AWS academy permite un máximo de **9 instancias corriendo** (que no creadas). Combinado con la restricción de las instancias de tipo large, como máximo se puede ejecutar el benchmark con **16 cores físicos** (si son instancias con procesadores Graviton [2]) o **16 threads** (8 cores físicos con 2 threads cada uno, si son instancias que utilizan otros procesadores [2]), ya que las instancias de tipo large sólo permiten hasta **2 vCPUS** y se cuenta con 8 nodos de cómputo corriendo **más el head node**. El benchmark sólo permite un número de procesos en potencia de 2, por lo que todas las posibilidades son con 1, 2, 4, 8 y 16 cores.

Otras limitaciones a destacar:

- No se permiten instancias de subasta (sólo instancias bajo demanda).
- Un total de 32 vCPUs entre todas las instancias.
- 100 GB de general purpose SSD (gp2, gp3)
- Si en algún momento se corren 20 o más instancias a la vez, la cuenta se desactivará automáticamente, borrando todos los recursos.

1.2 Kernel comunicaciones

Para este experimento se ha escogido el kernel **CG**. Es un kernel centrado en las comunicaciones **punto a punto**, que transfiere grandes cantidades de datos por la red (en torno a GB, lo que pondrá a prueba los retardos en la transmisión) [3]. También se ponen a prueba las comunicaciones **irregulares a larga distancia** (debido a que se trabaja con matrices dispersas, la cantidad de datos en cada envío/recibo de datos varia según el número de valores no nulos de la matriz) [4].

En ejecuciones de este kernel con los clústeres funcionando, no ha sido posible ejecutar este kernel con el tamaño D, ya que los recursos del clúster no son suficientes para abordar el tamaño del problema (segmentation fault). Por este motivo, no ha quedado más remedio que ejecutar el kernel en clase C.

1.2.1 Cálculo de costes

Esta sección determina los costes del número de instancias, el tipo de cada una y la duración total del workload.

Tipo de instancias

La calculadora de costes de Amazon [5] permite encontrar y comparar los costes de cada tipo de instancia fácilmente, filtrando según los recursos que se necesitan.

NOTA: todos los tipos de instancias y precios comentados pertenecen a la región de **North Virginia**. La disponibilidad de las instancias y el precio pueden variar según la región.

Se ha escogido el tipo de instancia **c5n.large** [6], ya que es el tipo de instancia de computación más económica que ofrece los siguientes parámetros:

- 2 vCPUS (1 core físico con 2 threads [2]) Intel(R) Xeon(R) Platinum 8259CL CPU @ 2.50GHz
- 5,25 GiB de memoria.
- **Hasta 25 Gbps de red.** La versión original (c5 sin la n) permite hasta 10 Gbps de red. Para la ejecución del kernel intensivo en comunicaciones se escogió la versión n para obtener un mejor rendimiento en el benchmark asumiendo un coste superior (la c5.large cuesta 0,085 USD la hora y la c5n.large cuesta 0,108 USD la hora).

Existen generaciones de instancias de cómputo posteriores, como las instancias c6in.large (0,1134 USD la hora), que también alcanzan los 25 Gbps, pero como el benchmark también se enfoca en el hardware de red, no es necesario invertir más dinero en ejecutar este kernel con la CPU más puntera existente en AWS.

Otra posibilidad de tipo de instancia era la **c6gn.large** [7], que ofrece un mejor rendimiento [8] y por un precio más económico (0,0864 USD la hora) o la instancia **c7gn.large**, que ofrece **hasta 30 Gbps de bandwidth** (por contraparte es más cara, 0,1323 USD la hora) [9]. Sin embargo, estas instancias incorporan la CPU AWS Graviton2 (c6g) y AWS Graviton3 (c7g) [10], que posee una ISA de ARM, **incompatible con la AMI del clúster**, que únicamente soporta la arquitectura x86_64.

Tiempo de ejecución

Para estimar el tiempo de ejecución del benchmark, puede usarse como referencia los tiempos de ejecución de un clúster que ejecutó el mismo kernel con la misma clase. Sin embargo, los últimos registros son de una máquina Cray Y-MP con un procesador para tamaños de máximo clase B en el año 1994 [4]. Por simpleza, se va a asumir que el kernel CG tarda 5 minutos en ejecutarse en su versión secuencial, y que el Speed up al subir el n° de cores (siguiendo potencias de 2), sube en 1 (comenzando con 1 para 2 cores). Se van a realizar 5 pruebas de 1, 2, 4, 8 y 16 cores, cada prueba de 5 iteraciones, por lo que en total se estima que el kernel tardará **77 minutos** ($5 \cdot 5 + 5 \cdot 5 \cdot 1 + 5 \cdot 5 \cdot 1/2 + 5 \cdot 5 \cdot 1/3 + 5 \cdot 5 \cdot 1/4$).

Coste estimado

Para realizar una aproximación del coste del experimento, se ha usado la calculadora de costes de AWS [5].

Se usaron un total de **18 instancias**:

- 2 instancias para los head nodes de los 2 clústeres. Se les asigna el tipo de instancia **t3.micro** [11] (0,0104 USD por hora, 2 vCPUS, 1 GiB de memoria y hasta 5 Gigabit de red).
- 16 nodos de cómputo, 8 para el clúster HPC y otras 8 para el clúster HTC, con el tipo de instancia **c5n.large**. Las restricciones del laboratorio no permite ejecutar más de 9 instancias a la vez, de ahí el motivo de escoger este número de instancias para los 2 clústeres.

Si se hacen cuentas:

- 8 instancias de cómputo x 0,108 USD por hora de la instancia x 1,28 horas al mes (77 minutos) x 2 clústers = **2,212 USD**
- 2 instancias a modo de head node x 0,0104 USD por hora de la instancia x 1,28 horas al mes (77 minutos) = **0,027 USD**

En total el coste estimado es de $2,212 + 0,027 = 2,239$ USD.

1.2.2 Despliegue

Finalmente se desplegaron los clústeres HTC y HPC mediante stacks de CloudFormation como se puede apreciar en las figuras 1.1 y 1.3. En las figuras 1.2 y 1.4 se distingue el tipo de clúster por la zona de disponibilidad de los nodos de cómputo. Para el clúster de HTC se despliegan las instancias en varias zonas de disponibilidad (las instancias de tipo c5n.large sólo permiten desplegarse en las zonas de disponibilidad: us-east-1a, us-east-1b, us-east-1c, us-east-1d, us-east-1f), mientras que en el clúster de HPC los nodos de cómputo se despliegan en la misma zona de disponibilidad que el head node (como se puede ver en la figura 1.4 se despliegan en us-east-1a).

1.3 Kernel computación

Para el kernel de computación se escogerá el kernel MG, que es uno de los kernels que menos comunicaciones MPI realiza [3]. Para un tamaño del problema de clase D, el kernel necesitaría **como mínimo 8 GB de memoria** para ejecutar el benchmark en punto flotante de 64 bits ($1024 \times 1024 \times 1024 \times 8 / 1024 \times 1024 \times 1024 = 8$ GB) [12], que es una cantidad de memoria superior a la que pueden ofrecer las instancias de tipo large. Por lo que la única opción es ejecutar el benchmark de clase C con el tipo de instancias que el laboratorio permite (tamaño large).

Los 2 clústeres de este experimento serán de tipo HPC. De este modo todas las instancias se encontrarán en la misma zona de disponibilidad, reduciendo la latencia en las comunicaciones (el objetivo es intentar empañar lo mínimo posible los tiempos de computación con los de comunicaciones).

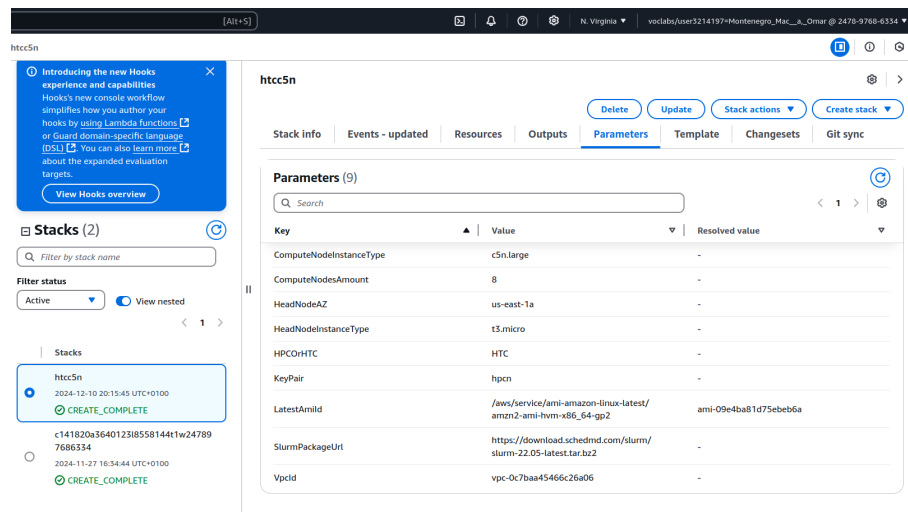


Figura 1.1: Stack de CloudFormation para el clúster HTC

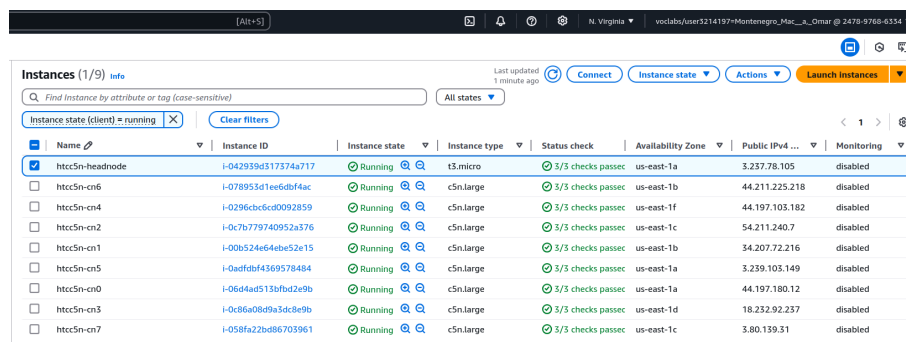


Figura 1.2: Panel de EC2 con las instancias del clúster HTC

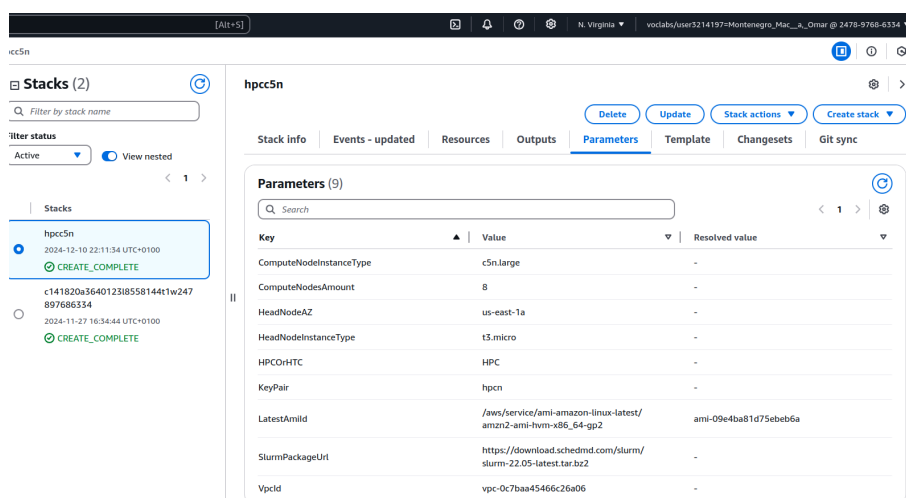


Figura 1.3: Stack de CloudFormation para el clúster HPC

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 ...	Monitoring
hpec5n-cn2	i-04fec9f179780714	Running	c5n.large	3/3 checks passed	us-east-1a	3.231.215.242	disabled
hpec5n-headnode	i-0d9730ee9f9d5b367	Running	t3.micro	3/3 checks passed	us-east-1a	3.89.81.164	disabled
hpec5n-cn0	i-0eb7695f5dc22745	Running	c5n.large	3/3 checks passed	us-east-1a	3.237.60.121	disabled
hpec5n-cn6	i-01ba0f05f6c9358e	Running	c5n.large	3/3 checks passed	us-east-1a	44.223.77.253	disabled
hpec5n-cn5	i-074aab4d80604f6fd	Running	c5n.large	3/3 checks passed	us-east-1a	44.195.23.40	disabled
hpec5n-cn3	i-0c5e4c89ba8e98d53	Running	c5n.large	3/3 checks passed	us-east-1a	3.238.108.65	disabled
hpec5n-cn4	i-06a5ab284719fc5d8	Running	c5n.large	3/3 checks passed	us-east-1a	3.237.239.228	disabled
hpec5n-cn7	i-094590bf41b32cbf	Running	c5n.large	3/3 checks passed	us-east-1a	3.238.69.9	disabled
hpec5n-cn1	i-0ac1c5c6f6780029	Running	c5n.large	3/3 checks passed	us-east-1a	44.197.119.116	disabled

Figura 1.4: Panel de EC2 con las instancias del clúster HPC

1.3.1 Cálculo de costes

Tipo de instancias

Para el clúster de propósito general, hay 2 tipos de familias a tener en cuenta:

- Familia T: son instancias pensadas para satisfacer picos momentáneos de demanda, el resto del tiempo se encuentran infrautilizadas (un ejemplo es un servidor web). Se basan en un sistema de créditos del uso de la CPU[13], que otorga créditos cuando la demanda se encuentra por debajo de un baseline y estos se gastan cuando se supera el baseline por un periodo de tiempo (1 crédito por minuto). Según el tamaño de la instancia, más créditos se otorgarán, para más información puede consultarse la correspondiente documentación de Amazon [14]. **No son el tipo de instancias adecuadas para el workload del experimento.**
- Familia M: no siguen el sistema de créditos de la familia T. Estas instancias otorgan un balance entre computación, memoria y red para workloads con cierta intensidad (big data en tiempo real, bases de datos de tamaño mediano) [15]. Son el tipo de instancias que se escogerán para este experimento.

Para el clúster de propósito general, se eligieron instancias del tipo m5a.large [16] (0,086 USD la hora). El objetivo era escoger una instancia al menos una generación anterior a la de cómputo. Entre las características de las instancias m5a destacan:

- 2 vCPUs (1 core físico con 2 threads [2]) AMD EPYC 7000.
- 8 GiB de memoria.
- 10 Gbps de red.

Para el clúster de rendimiento se ha optado por escoger las instancias c7i.large. Dentro de la familia existen las instancias c7i-flex.large, pero Amazon las recomienda para aplicaciones que **no** utilicen al completo los recursos de cómputo [17]. El motivo principal de elección de estas instancias se debe al precio muy similar respecto a las instancias m5a (0,089 USD de la instancia c7i.large respecto a 0,086 USD de la instancia m5a.large), lo que sería una comparación interesante en cuanto al balance rendimiento-precio. Entre las características de las instancias c7i.large destacan [17]:

- 2 vCPUs (1 core físico con 2 threads [2]) 4th Generation Intel Xeon Scalable (Sapphire Rapids).

- 4 GiB de memoria (ratio 2:1 de memoria a vCPU [17]).
- Hasta 12,5 Gbps

También se han hecho algunas pruebas con las instancias c6id.large (0,1134 USD la hora) [18]. Estas instancias se despliegan en servidores **con discos SSD NVMe directamente conectados**, lo que podría ser interesante para ejecutar aplicaciones CFD, ya que estas suelen generar ficheros temporales de gran tamaño durante la computación. Con los discos NVMe se puede reducir considerablemente el tiempo de acceso y de escritura a estos ficheros.

Tiempo de ejecución

Considerando las suposiciones para el kernel CG y teniendo en cuenta que el kernel MG se ejecuta varios minutos más rápido que el kernel CG [4], se asumirá que el kernel MG tarda 2 minutos en ejecutarse en su versión secuencial. En total el kernel MG se estima que tardará **31 minutos** ($2 \cdot 5 + 2 \cdot 5 \cdot 1 + 2 \cdot 5 \cdot 1/2 + 2 \cdot 5 \cdot 1/3 + 2 \cdot 5 \cdot 1/4$).

Coste estimado

Se usarán un total de 18 instancias:

- 2 instancias para los head nodes de los 2 clústeres. Se les asigna el tipo de instancia **t3.micro** (0,0104 USD por hora, 2 vCPUS, 1 GiB de memoria y hasta 5 Gigabit de red).
- 16 nodos de cómputo, 8 por clúster HPC. 1 clúster HPC utiliza instancias del tipo m5a.large (0,086 USD la hora) y el otro usará instancias del tipo c7i.large (0,089 USD la hora).

Si se realizan los cálculos:

- 8 instancias de cómputo m5a x 0,086 USD por hora de la instancia x 0,52 horas al mes (31 minutos) + 8 instancias de cómputo c7i x 0,089 USD por hora de la instancia x 0,52 horas al mes (31 minutos) = **0,728 USD**
- 2 instancias a modo de head node x 0,0104 USD por hora de la instancia x 0,52 horas al mes (31 minutos) = **0,011 USD**

En total el coste estimado es de $0,728 + 0,011 = \mathbf{0,739 \text{ USD}}$.

1.3.2 Despliegue

Los 2 clústeres HPC se desplegaron con las instancias m5a y c7i, como se muestra en las figuras 1.5 y 1.8 las instancias se han creado en la misma zona de disponibilidad (us-east-1a). En las figuras 1.6 y 1.7 se muestra el stack de CloudFormation con los recursos de cada clúster.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 ...	Monitoring	Security group name	Key name
hpcmsa-cn1	i-08e4698655b7102ff	Running	m5a.large	3/3 checks passed	us-east-1a	44.202.234.59	disabled	hpcmsa-SecurityGroup-...	hpcn
hpcmsa-cn4	i-07734515e001a10a9	Running	m5a.large	3/3 checks passed	us-east-1a	3.235.47.1	disabled	hpcmsa-SecurityGroup-...	hpcn
hpcmsa-cn0	i-02ee66a4113064107	Running	m5a.large	3/3 checks passed	us-east-1a	44.196.158.93	disabled	hpcmsa-SecurityGroup-...	hpcn
hpcmsa-cn6	i-047c3574d0132663c	Running	m5a.large	3/3 checks passed	us-east-1a	3.236.89.142	disabled	hpcmsa-SecurityGroup-...	hpcn
hpcmsa-headnode	i-0c85683b605a7354	Running	t3.micro	3/3 checks passed	us-east-1a	44.214.117.37	disabled	hpcmsa-SecurityGroup-...	hpcn
hpcmsa-cn5	i-00996b465a5b4796b	Running	m5a.large	3/3 checks passed	us-east-1a	44.204.253.90	disabled	hpcmsa-SecurityGroup-...	hpcn
hpcmsa-cn7	i-021788b3123c3c19	Running	m5a.large	3/3 checks passed	us-east-1a	3.80.163.69	disabled	hpcmsa-SecurityGroup-...	hpcn
hpcmsa-cn1	i-075f6c5c4432321	Running	m5a.large	3/3 checks passed	us-east-1a	44.195.197.34	disabled	hpcmsa-SecurityGroup-...	hpcn
hpcmsa-cn2	i-05c670e642a5ee3a	Running	m5a.large	3/3 checks passed	us-east-1a	3.218.206.243	disabled	hpcmsa-SecurityGroup-...	hpcn

Figura 1.5: Clúster HPC usando instancias de propósito general

Key	Value	Resolved value
ComputeNodeInstanceType	m5a.large	-
ComputeNodesAmount	8	-
HeadNodeAZ	us-east-1a	-
HeadNodeInstanceType	t3.micro	-
HCPCoreHTC	HPC	-
KeyPair	hpcn	-
LatestAmiId	/aws/service/ami-amazon-linux-latest/amzn2-ami-hvm-x86_64-gp2	ami-09e4ba1d75eb6b6a
SlurmPackageUrl	https://download.schedmd.com/slurm/slurm-22.05-latest.tar.bz2	-
VpcId	vpc-0c7baa45466c26a06	-

Figura 1.6: Clúster desplegado con instancias m5a a partir CloudFormation

Key	Value	Resolved value
ComputeNodeInstanceType	c7i.large	-
ComputeNodesAmount	8	-
HeadNodeAZ	us-east-1a	-
HeadNodeInstanceType	t3.micro	-
HCPCoreHTC	HPC	-
KeyPair	hpcn	-
LatestAmiId	/aws/service/ami-amazon-linux-latest/amzn2-ami-hvm-x86_64-gp2	ami-09e4ba1d75eb6b6a
SlurmPackageUrl	https://download.schedmd.com/slurm/slurm-22.05-latest.tar.bz2	-
VpcId	vpc-0c7baa45466c26a06	-

Figura 1.7: Clúster desplegado con instancias c7i a partir CloudFormation

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 ...	Monitoring	Security group name	Key name
hpc7i-cn1	i-08696b7590d960a7a	Running	c7i.large	3/3 checks passed	us-east-1a	44.222.75.171	disabled	hpc7i-SecurityGroup-b...	hpcn
hpc7i-cn2	i-01062009540bb425	Running	c7i.large	3/3 checks passed	us-east-1a	10.204.23.7	disabled	hpc7i-SecurityGroup-b...	hpcn
hpc7i-cn4	i-00e549b153063497d	Running	c7i.large	3/3 checks passed	us-east-1a	3.238.8.100	disabled	hpc7i-SecurityGroup-b...	hpcn
hpc7i-cn5	i-01f8e458314084d79	Running	c7i.large	3/3 checks passed	us-east-1a	44.195.79.250	disabled	hpc7i-SecurityGroup-b...	hpcn
hpc7i-cn7	i-0d4f4e43c64d8524b	Running	c7i.large	3/3 checks passed	us-east-1a	54.85.84.220	disabled	hpc7i-SecurityGroup-b...	hpcn
hpc7i-cn3	i-08f6ed302725f0f07	Running	c7i.large	3/3 checks passed	us-east-1a	3.235.89.151	disabled	hpc7i-SecurityGroup-b...	hpcn
hpc7i-cn6	i-0408c1e5446309ca0	Running	c7i.large	3/3 checks passed	us-east-1a	44.198.181.98	disabled	hpc7i-SecurityGroup-b...	hpcn
hpc7i-headnode	i-039731c77f4f25c7c	Running	t3.micro	3/3 checks passed	us-east-1a	98.84.120.223	disabled	hpc7i-SecurityGroup-b...	hpcn
hpc7i-cn0	i-0b350060714dbd00	Running	c7i.large	3/3 checks passed	us-east-1a	3.236.203.133	disabled	hpc7i-SecurityGroup-b...	hpcn

Figura 1.8: Clúster HPC usando instancias c7i

Resultados del experimento

2.1 Métricas para cada kernel

En este experimento se usarán las siguientes métricas para analizar la escalabilidad a partir de los resultados:

- Tiempo de ejecución
- Speed up (el resultado ideal sería igual al n° de cores)
- Eficiencia paralela (el resultado ideal sería igual a 1)

Todas las métricas utilizan el tiempo de ejecución transcurrido **en segundos**.

NOTA: para el cálculo del speed up y de la eficiencia paralela se asume que el tiempo secuencial **es el tiempo de ejecución del algoritmo paralelo con 1 core** (y no el tiempo de ejecución del código secuencial).

El experimento se basa en un **strong scaling**, es decir, se ejecuta el mismo problema **con el mismo tamaño** (clase C para los 2 kernels escogidos) variando de 1, 2, 4, 8 a 16 cores.

2.2 Factores primarios:

A continuación se muestra el software utilizado en el experimento y sus correspondientes versiones.

- NPB-MPI versión 3.4.3
- gcc versión 7.3.1, con flags de optimización -O3
- Open MPI versión 4.0.1
- slurm versión 22.05.11

2.3 Resultados del kernel de comunicación CG

2.3.1 Tiempo promedio

Clúster HTC:

Tiempo promedio (s)	1 core	2 cores	4 cores	8 cores	16 cores
clúster HTC	227,018	201,196	103,616	32,714	22,228
clúster HPC	229,662	202,92	106,176	31,486	18,156

Táboa 2.1: Tiempo promedio de ejecución del kernel CG en segundos

Speed up	2 cores	4 cores	8 cores	16 cores
clúster HTC	1,128	2,191	6,939	10,213
clúster HPC	1,132	2,163	7,294	12,649

Táboa 2.2: Speed up del kernel CG en segundos

- 1 core: $(226,98+226,96+227,10+227,12+226,93)/5 = \mathbf{227,018 \text{ segundos}}$
- 2 cores: $(199,69+199,69+199,85+206,73+200,02)/5 = \mathbf{201,196 \text{ segundos}}$
- 4 cores: $(103,76+103,61+103,37+103,60+103,74)/5 = \mathbf{103,616 \text{ segundos}}$
- 8 cores: $(32,54+32,79+32,67+32,80+32,77)/5 = \mathbf{32,714 \text{ segundos}}$
- 16 cores: $(22,88+21,53+22,33+23,05+21,35)/5 = \mathbf{22,228 \text{ segundos}}$

Clúster HPC:

- 1 core: $(230,03+229,70+229,75+229,35+229,48)/5 = \mathbf{229,662 \text{ segundos}}$
- 2 cores: $(201,91+202,53+204,01+202,92+203,23)/5 = \mathbf{202,92 \text{ segundos}}$
- 4 cores: $(104,66+109,57+106,79+104,61+105,25)/5 = \mathbf{106,176 \text{ segundos}}$
- 8 cores: $(31,38+31,65+31,48+31,47+31,45)/5 = \mathbf{31,486 \text{ segundos}}$
- 16 cores: $(18,09+18,20+18,21+18,15+18,13)/5 = \mathbf{18,156 \text{ segundos}}$

2.3.2 Speed Up

Clúster HTC:

- 2 cores: $227,018/201,196 = \mathbf{1,128}$
- 4 cores: $227,018/103,616 = \mathbf{2,191}$
- 8 cores: $227,018/32,714 = \mathbf{6,939}$
- 16 cores: $227,018/22,228 = \mathbf{10,213}$

Clúster HPC:

- 2 cores: $229,662/202,92 = \mathbf{1,132}$

Eficiencia paralela	2 cores	4 cores	8 cores	16 cores
clúster HTC	0,564	0,548	0,867	0,638
clúster HPC	0,566	0,541	0,912	0,791

Táboa 2.3: Eficiencia paralela del kernel CG en segundos

- 4 cores: $229,662/106,176 = 2,163$
- 8 cores: $229,662/31,486 = 7,294$
- 16 cores: $229,662/18,156 = 12,649$

2.3.3 Eficiencia paralela

Clúster HTC:

- 2 cores: $227,018/(2*201,196) = 0,564$
- 4 cores: $227,018/(4*103,616) = 0,548$
- 8 cores: $227,018/(8*32,714) = 0,867$
- 16 cores: $227,018/(16*22,228) = 0,638$

Clúster HPC:

- 2 cores: $229,662/(2*202,92) = 0,566$
- 4 cores: $229,662/(4*106,176) = 0,541$
- 8 cores: $229,662/(8*31,486) = 0,912$
- 16 cores: $229,662/(16*18,156) = 0,791$

2.3.4 Análisis

En la figura 2.1 puede verse como el clúster HTC consigue terminar un par de segundos antes respecto al clúster HPC. Este comportamiento se mantiene hasta los 4 cores. A partir de este punto, el clúster HPC empieza a terminar antes su ejecución respecto al clúster HTC (con poco más de 1 segundo de diferencia), con 16 cores la diferencia se acentúa a favor del clúster de HPC (4 segundos de diferencia).

Si se hace incapié en los resultados del Speed up de la figura 2.2, se puede apreciar de forma más visual el comportamiento comentado para el tiempo de ejecución. En este caso, al aumentar el número de cores, aumenta el Speed up de forma más pronunciada en el clúster de HPC. Esto puede deberse a que la carga del benchmark se distribuye en más instancias al aumentar el número de cores, por lo que tiene que realizar una mayor distribución de datos entre estas. Para el clúster HPC no supone mucho problema, ya que todas las instancias se encuentran en la misma zona de disponibilidad, por lo tanto, las instancias se encuentran relativamente contiguas entre sí (en orden de metros). Sin embargo, el

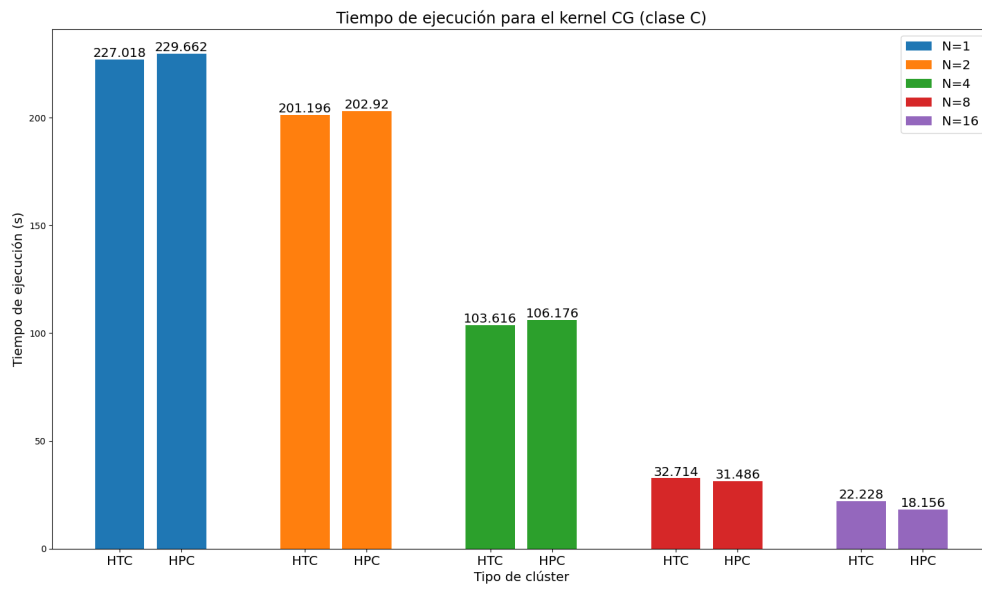


Figura 2.1: Tiempo de ejecución del kernel CG

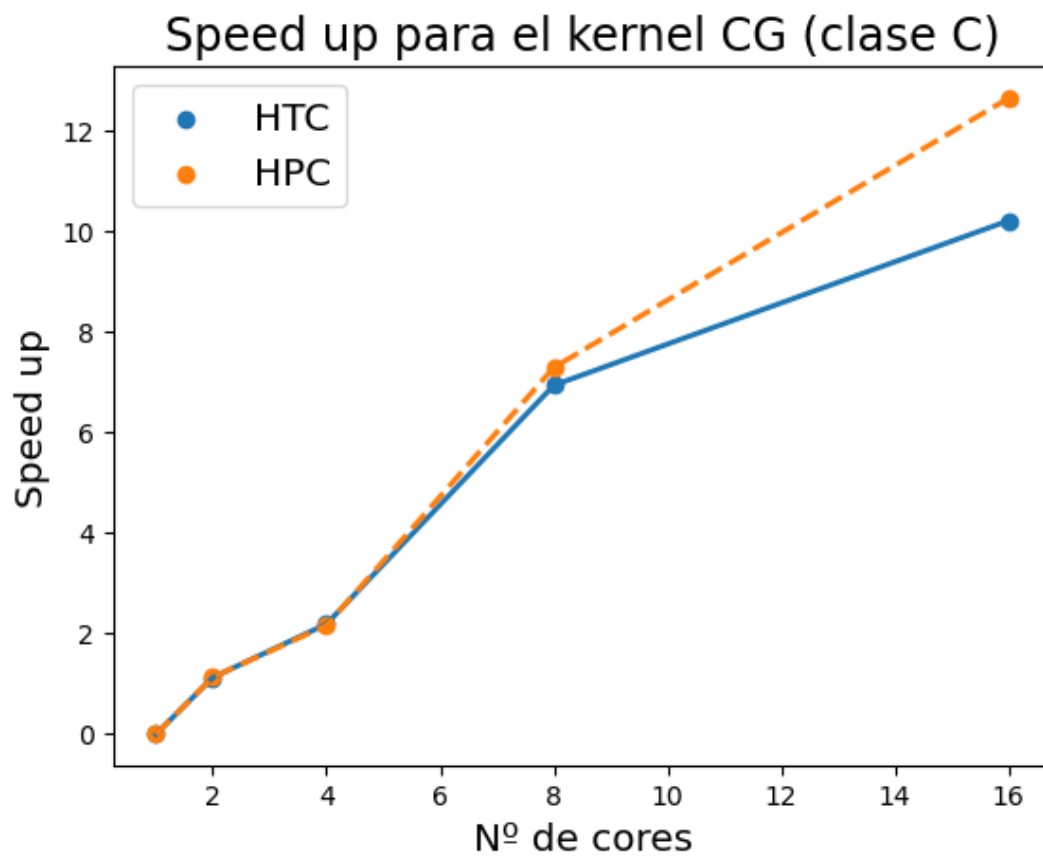


Figura 2.2: Speed up del kernel CG

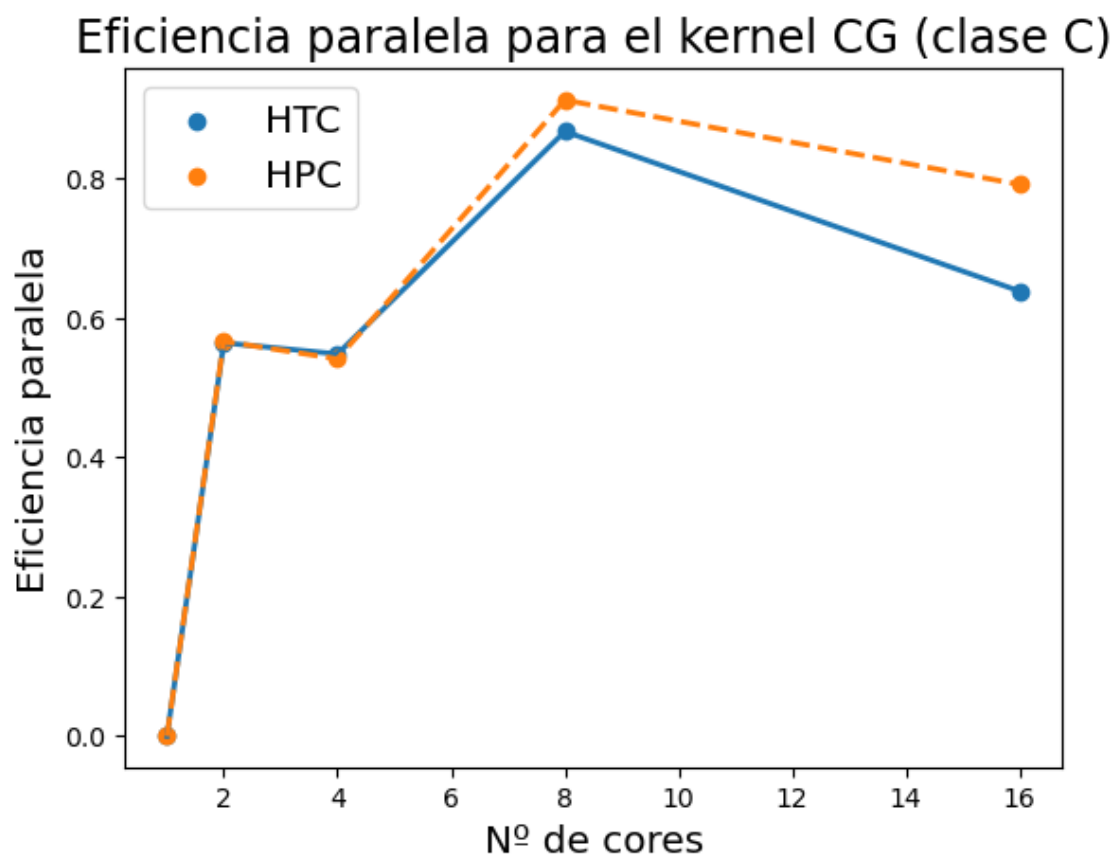


Figura 2.3: Eficiencia paralela del kernel CG

Tiempo promedio (s)	1 core	2 cores	4 cores	8 cores	16 cores
clúster m5a	46,412	38,836	20,434	8,916	5,146
clúster c7i	36,14	27,988	15,224	7,724	4,69

Táboa 2.4: Tiempo promedio de ejecución del kernel MG en segundos

clúster HTC distribuye sus instancias entre varias zonas de disponibilidad, por lo que las latencias de transmisión se agravan debido a la distancia que separa a las instancias (en orden de kilómetros), por lo que el Speed up tiende a crecer cada vez menos (incluso llegando al punto de decrecer si el benchmark se ejecuta con muchos cores).

En el caso de la eficiencia paralela en la figura 2.3, se muestra para ambos clústers que su pico de eficiencia se encuentra en los 8 cores. Aunque según los resultados de Speed up el clúster HPC vaya en subida, la gráfica de eficiencia paralela (figura 2.3) indica que el clúster ya ha alcanzado su pico máximo de eficiencia y que a partir de ahí va a ir en declive. Por otro lado, el clúster HTC indica una mayor caída de la eficiencia respecto al clúster HPC, este fenómeno se debe al factor ya comentado con el Speed up.

Tras analizar los resultados, se ha determinado que el clúster de HTC **no escala correctamente** respecto al clúster HPC. Ya que según los resultados de Speed up y eficiencia paralela, el rendimiento de este clúster tiende a caer notoriamente. De todos modos, el workload ejecutado no sigue la filosofía de los clústeres HTC, ya que las aplicaciones HTC no incurren muchas comunicaciones entre procesos, por lo que la necesidad de una red de alto rendimiento **no es crítica** [19].

El tipo de instancia c5n.large ha tenido que influir positivamente en los resultados del kernel CG, especialmente para el clúster HTC. Si se hubiera escogido una instancia con menos de 25 Gbps, los resultados serían peores para el clúster de HTC.

2.4 Resultados kernel de computación MG

2.4.1 Tiempo promedio

Clúster m5a:

- 1 core: $(46,46+46,36+46,45+46,37+46,42)/5 = \mathbf{46,412 \text{ segundos}}$
- 2 cores: $(38,81+38,70+38,88+38,81+38,98)/5 = \mathbf{38,836 \text{ segundos}}$
- 4 cores: $(20,41+20,54+20,39+20,35+20,48)/5 = \mathbf{20,434 \text{ segundos}}$
- 8 cores: $(8,89+8,95+8,86+8,82+9,06)/5 = \mathbf{8,916 \text{ segundos}}$
- 16 cores: $(5,19+5,27+5,09+5,07+5,11)/5 = \mathbf{5,146 \text{ segundos}}$

Clúster c7i:

- 1 core: $(36,17+36,17+36,13+36,09+36,4)/5 = \mathbf{36,14 \text{ segundos}}$
- 2 cores: $(27,96+28,00+27,98+28,00+28,00)/5 = \mathbf{27,988 \text{ segundos}}$

Speed up	2 cores	4 cores	8 cores	16 cores
clúster m5a	1,195	2,271	5,205	9,019
clúster c7i	1,291	2,374	4,679	7,706

Táboa 2.5: Speed up del kernel MG en segundos

Eficiencia paralela	2 cores	4 cores	8 cores	16 cores
clúster m5a	0,598	0,568	0,568	0,564
clúster c7i	0,646	0,593	0,585	0,482

Táboa 2.6: Eficiencia paralela del kernel MG en segundos

- 4 cores: $(15,27+15,21+15,26+15,19+15,19)/5 = \mathbf{15,224 \text{ segundos}}$
- 8 cores: $(7,74+7,70+7,68+7,72+7,78)/5 = \mathbf{7,724 \text{ segundos}}$
- 16 cores: $(4,67+4,65+4,82+4,48+4,83)/5 = \mathbf{4,69 \text{ segundos}}$

2.4.2 Speed Up

Clúster m5a:

- 2 cores: $46,412/38,836 = \mathbf{1,195}$
- 4 cores: $46,412/20,434 = \mathbf{2,271}$
- 8 cores: $46,412/8,916 = \mathbf{5,205}$
- 16 cores: $46,412/5,146 = \mathbf{9,019}$

Clúster c7i:

- 2 cores: $36,14/27,988 = \mathbf{1,291}$
- 4 cores: $36,14/15,224 = \mathbf{2,374}$
- 8 cores: $36,14/7,724 = \mathbf{4,679}$
- 16 cores: $36,14/4,69 = \mathbf{7,706}$

2.4.3 Eficiencia paralela

Clúster m5a:

- 2 cores: $46,412/(2*38,836) = \mathbf{0,598}$
- 4 cores: $46,412/(4*20,434) = \mathbf{0,568}$

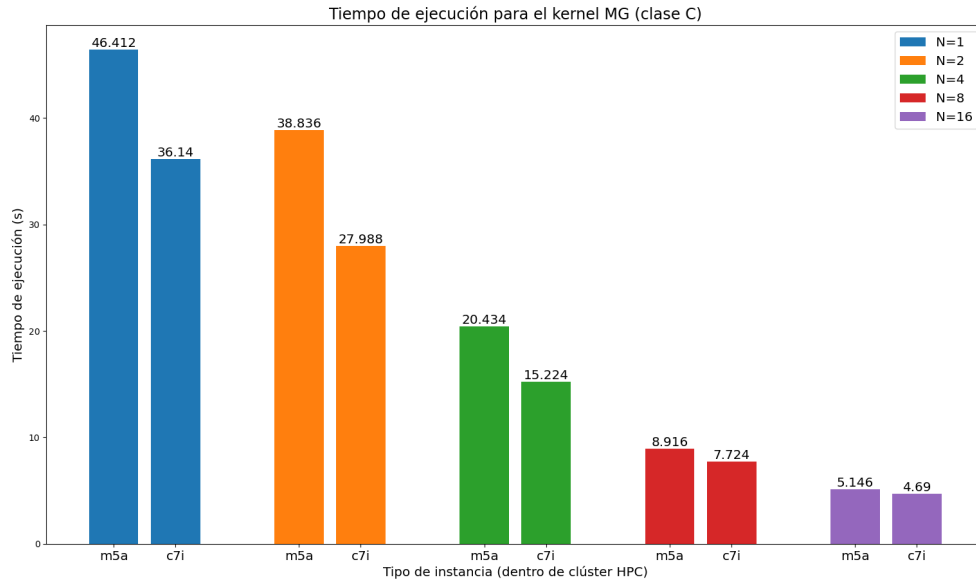


Figura 2.4: Tiempo ejecución del kernel MG

- 8 cores: $46,412/(8*8,916) = \mathbf{0,568}$
- 16 cores: $46,412/(16*5,146) = \mathbf{0,564}$

Clúster c7i:

- 2 cores: $36,14/(2*27,988) = \mathbf{0,646}$
- 4 cores: $36,14/(4*15,224) = \mathbf{0,593}$
- 8 cores: $36,14/(8*7,724) = \mathbf{0,585}$
- 16 cores: $36,14/(16*4,69) = \mathbf{0,482}$

2.4.4 Análisis

En la figura 2.4 se ve una diferencia amplia a favor del clúster c7i. Al aumentar el número de cores, la diferencia entre clústeres respecto al tiempo de ejecución va disminuyendo. Esto se debe a que el margen de mejora de c7i se vuelve inferior al de m5a. Para un problema de tamaño superior, estas diferencias se ampliarían a favor del clúster c7i, que no es mucho más caro que el clúster m5a (0,089 USD de la instancia c7i.large respecto a 0,086 USD de la instancia m5a.large), por lo que la instancia m5a en cuanto a rendimiento-precio **no merece la pena** para este tipo de workloads.

En cuanto al Speed up (figura 2.5), se aprecia que para el clúster m5a aumenta en mayor grado que para el clúster c7i a partir de los 8 cores. Como ya se mencionó, el clúster c7i está llegando al límite máximo de paralelización del kernel (en este caso habría que optar por un problema más grande, pero como ya se vió esto no es posible).

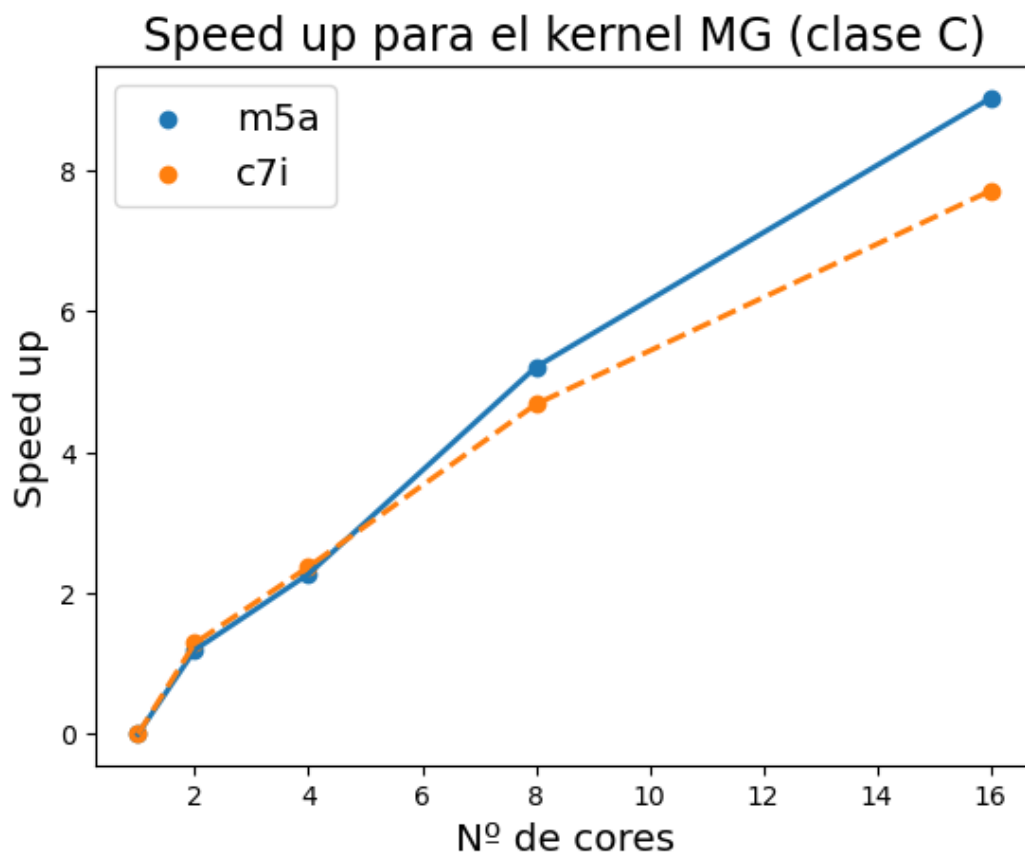


Figura 2.5: Speed up del kernel MG

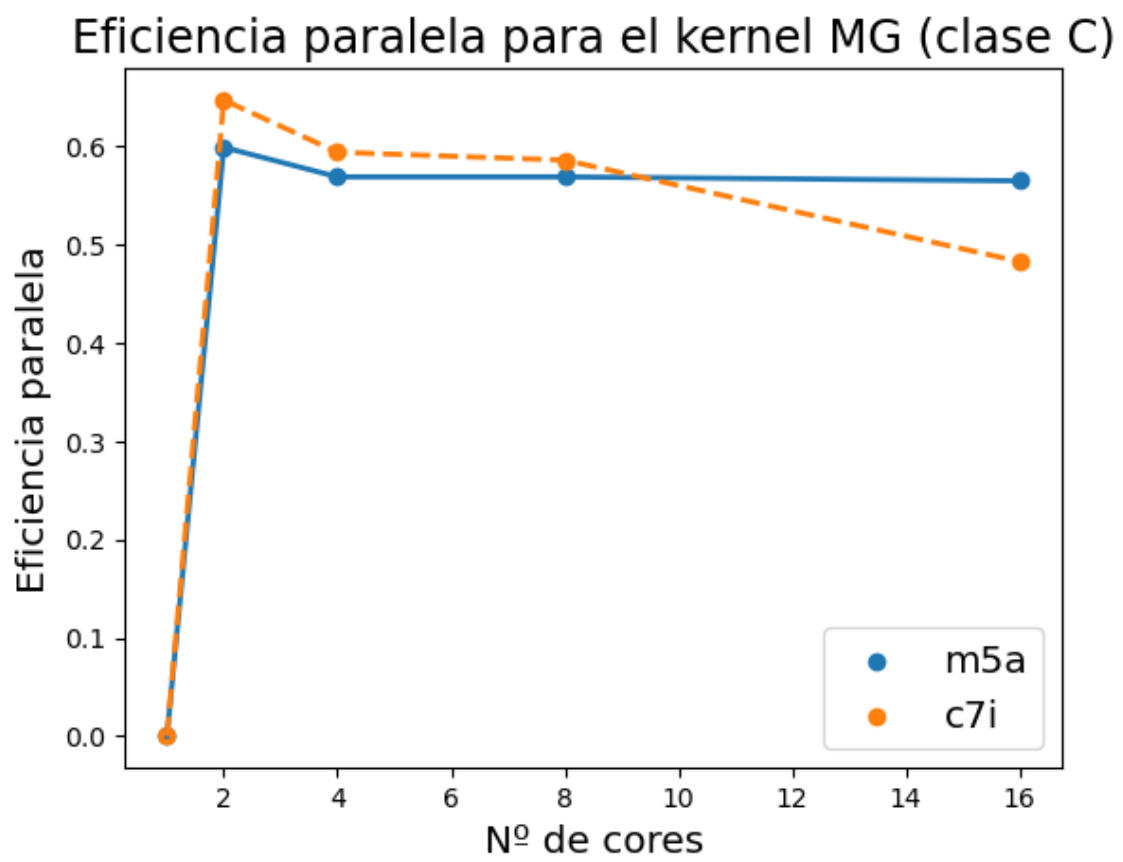


Figura 2.6: Eficiencia paralela del kernel MG

Tiempo promedio (s)	1 core	4 cores
clúster c6id	30,156	14,086
clúster c7i	36,14	15,224

Táboa 2.7: Tiempo de ejecución promedio en el kernel MG para los clústeres c6id y c7i

En la figura 2.6 se muestra para el clúster m5a una estabilización de la eficiencia paralela, también para el clúster c7i hasta que alcanza los 8 cores. Esto indica que para el clúster m5a el Speed up progresa adecuadamente al aumentar los cores y para el clúster c7i disminuye debido al límite de paralelismo ya comentado.

Por último, es interesante comentar los resultados del kernel MG en un tipo de instancia de una generación **anterior** a las instancias c7i. La instancia c6id.large [18] ha demostrado otorgar **mejor rendimiento** que las instancias de tipo c7i.large como se muestra en la tabla 2.7. El principal factor puede deberse a que el kernel MG se beneficia de los discos SSD NVMe de este tipo de instancias, frente a los volúmenes EBS gp2 y gp3. En cuanto a los procesadores que incorporan los 2 tipos de instancias se demuestra que no hay tanta diferencia que la que existe entre los discos NVMe y los gp2 y gp3 de EBS para aplicaciones como la del kernel MG. A pesar de esto, las instancias c6id.large **son más caras** (0,1134 USD la hora respecto a 0,089 USD de las c7i).

Tiempo de ejecución del clúster c6id:

- 1 core: $(30,04+30,09+30,35+30,16+30,14)/5 = \mathbf{30,156 \text{ segundos}}$
- 4 cores: $(14,05+14,14+14,10+14,13+14,01)/5 = \mathbf{14,086 \text{ segundos}}$

2.5 Conclusiones

Se ha realizado un análisis de la escalabilidad de aplicaciones de HPC en AWS usando instancias optimizadas para cálculo y de propósito general dentro de las limitaciones de AWS Academy desde el punto de vista de las comunicaciones (kernel CG) y de la computación (kernel MG).

La afinidad de las instancias es un factor a tener en cuenta a la hora de ejecutar aplicaciones intensivas en comunicaciones (en este caso, el kernel CG), ya que al desplegar instancias en diferentes zonas de disponibilidad, las distancias entre estas se miden en orden de kilómetros (caso del clúster HTC). Mientras que al desplegar un clúster en una única zona de disponibilidad, todos sus nodos se encuentran en el mismo centro de datos, estando distanciadas entre sí por unos metros (caso del clúster HPC). Este factor se hace notar a medida que se aumenta el número de cores del clúster, ya que implica lanzar más instancias y realizar más comunicaciones. El tipo de instancia c5n.large ha obtenido buenos resultados en el clúster HTC, que ha sabido mantenerse en términos de rendimiento respecto al clúster HPC. Si se hubiera escogido un tipo de instancia con menos de 25 Gbps de bandwidth, las diferencias se habrían agravado en contra del clúster HTC.

Las instancias de propósito general no son adecuadas para las aplicaciones intensivas en computación, ya que están pensadas para ofrecer buenas prestaciones en todos sus recursos sin centrarse en la potencia de cómputo. En cambio, las instancias optimizadas para cómputo han demostrado otorgar

mejores resultados por un precio muy similar para workloads en la misma línea que el kernel MG. De todos modos, ambos tipos de instancias han demostrado una buena progresión en el Speed up y han logrado mantener los valores de la eficiencia paralela. A excepción de las últimas pruebas con las instancias c7i, cuyos resultados han empeorado por estar alcanzando el límite de paralelismo. Adicionalmente, se ha probado el rendimiento del kernel MG con instancias que incorporan SSDs NVMe (c6id), que han conseguido superar en rendimiento a instancias con procesadores de generaciones posteriores (c7i).

Bibliografía

- [1] A. L. Lab, “Service usage and other restrictions,” 2024, consultado o 17 de diciembre de 2024. [En línea]. Disponible en: <https://labs.vocareum.com/web/3214197/3640123.0/ASNLB/public/docs/lang/en-us/README.html?vockey=05b471cc45b598244a754f6a61ee1312f543217cefa2a0c1d08c5f54c8666d71#services>
- [2] Amazon, “Opciones de cpu compatibles con los tipos de instancias de amazon ec2,” 2024, consultado o 17 de diciembre de 2024. [En línea]. Disponible en: https://docs.aws.amazon.com/es_es/AWSEC2/latest/UserGuide/cpu-options-supported-instances-values.html?ref=martysweet.co.uk#cpu-options-compute-optimized/
- [3] D. of Computer Science Florida State University Tallahassee FL 32306, “Communication characteristics in the nas parallel benchmarks,” 2002, consultado o 17 de diciembre de 2024. [En línea]. Disponible en: <http://www.cs.fsu.edu/~xyuan/paper/02pdcs.pdf>
- [4] D. B. E. B. J. B. D. B. R. C. L. D. R. F. S. F. P. F. T. L. R. S. H. S. V. Venkatakrishnan and S. Weeratunga, “The nas parallel benchmarks,” 2002, consultado o 17 de diciembre de 2024. [En línea]. Disponible en: <https://www.nas.nasa.gov/assets/nas/pdf/techreports/1994/rnr-94-007.pdf>
- [5] Amazon, “Create estimate: Configure amazon ec2,” 2024, consultado o 17 de diciembre de 2024. [En línea]. Disponible en: <https://calculator.aws/#/createCalculator/ec2-enhancement?key=new>
- [6] —, “Amazon ec2 c5 instances,” 2024, consultado o 17 de diciembre de 2024. [En línea]. Disponible en: <https://aws.amazon.com/ec2/instance-types/c5/>
- [7] —, “Amazon ec2 c6g instances,” 2024, consultado o 17 de diciembre de 2024. [En línea]. Disponible en: <https://aws.amazon.com/ec2/instance-types/c6g/>
- [8] L. AWS, “Aws ec2 instance comparison: C6g vs c5,” 2020, consultado o 17 de diciembre de 2024. [En línea]. Disponible en: <https://www.learnaws.org/2020/12/21/c6g-vs-c5/>
- [9] Amazon, “Amazon ec2 c7g instances,” 2024, consultado o 17 de diciembre de 2024. [En línea]. Disponible en: <https://aws.amazon.com/ec2/instance-types/c7g/>
- [10] CLOUDZERO, “What is aws graviton? here’s when to use it,” 2024, consultado o 17 de diciembre de 2024. [En línea]. Disponible en: <https://www.cloudzero.com/blog/aws-graviton/>

- [11] Amazon, “Amazon ec2 t3 instances,” 2024, consultado o 17 de decembro de 2024. [En liña]. Disponible en: <https://aws.amazon.com/ec2/instance-types/t3/>
- [12] N. A. S. N. Division, “Problem sizes and parameters in nas parallel benchmarks,” 2021, consultado o 17 de decembro de 2024. [En liña]. Disponible en: https://www.nas.nasa.gov/software/npb_problem_sizes.html
- [13] Amazon, “Conceptos clave para las instancias de rendimiento ampliables,” 2024, consultado o 17 de decembro de 2024. [En liña]. Disponible en: https://docs.aws.amazon.com/es_es/AWSEC2/latest/UserGuide/burstable-credits-baseline-concepts.html
- [14] —, “Amazon ec2 instance types,” 2024, consultado o 17 de decembro de 2024. [En liña]. Disponible en: https://docs.aws.amazon.com/es_es/AWSEC2/latest/UserGuide/burstable-credits-baseline-concepts.html
- [15] —, “Aws t3 vs. m5 instances: A side-by-side comparison,” 2024, consultado o 17 de decembro de 2024. [En liña]. Disponible en: <https://www.cloudzero.com/advisor/t3-vs-m5/>
- [16] —, “Amazon ec2 m5 instances,” 2024, consultado o 17 de decembro de 2024. [En liña]. Disponible en: <https://aws.amazon.com/ec2/instance-types/m5/>
- [17] —, “Amazon ec2 c7i and c7i-flex instances,” 2020, consultado o 17 de decembro de 2024. [En liña]. Disponible en: <https://aws.amazon.com/ec2/instance-types/c7i/>
- [18] —, “Amazon ec2 c6i instances,” 2024, consultado o 17 de decembro de 2024. [En liña]. Disponible en: <https://aws.amazon.com/ec2/instance-types/c6i/>
- [19] —, *AWS Well-Architected Framework — High Performance Computing Lens*, 1st ed. Extraído del contenido teórico de la asignatura, 2019.